

torch.fft.rfft2#

<https://pytorch.org/docs/stable/generated/torch.fft.rfft2.html>

Description:

Computes the 2-dimensional discrete Fourier transform of real input. Equivalent to `rfftn()` but FFTs only the last two dimensions by default. The FFT of a real signal is Hermitian-symmetric, $X[i, j] = \text{conj}(X[-i, -j])$, so the full `fft2()` output contains redundant information. `rfft2()` instead omits the negative frequencies in the last dimension. Supports `torch.half` on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions. input (Tensor) – the input tensor

Function Signature

```
torch.fft.rfft2(input, s=None, dim=(-2, -1), norm=None, *,  
out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> t = torch.rand(10, 10)
>>> rfft2 = torch.fft.rfft2(t)
>>> rfft2.size()
torch.Size([10, 6])
```

```
>>> fft2 = torch.fft.fft2(t)
>>> torch.testing.assert_close(fft2[..., :6], rfft2,
check_stride=False)
```

```
>>> two_ffts = torch.fft.fft(torch.fft.rfft(t, dim=1), dim=0)
>>> torch.testing.assert_close(rfft2, two_ffts,
check_stride=False)
```

Notes & Warnings

NOTE

Note Supports torch.half on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

Detailed Documentation

Documentation

Computes the 2-dimensional discrete Fourier transform of real input. Equivalent to `rfftn()` but FFTs only the last two dimensions by default.

The FFT of a real signal is Hermitian-symmetric, $X[i, j] = \text{conj}(X[-i, -j])$, so the full `fft2()` output contains redundant information. `rfft2()` instead omits the negative frequencies in the last dimension.

Supports torch.half on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

`input (Tensor)` – the input tensor

`s (Tuple[int], optional)` – Signal size in the transformed dimensions. If given, each dimension `dim[i]` will either be zero-padded or trimmed to the length `s[i]` before computing the real FFT. If a length `-1` is specified, no padding is done in that dimension. Default: `s = [input.size(d) for d in dim]`

`dim (Tuple[int], optional)` – Dimensions to be transformed. Default: last two dimensions.

`norm (str, optional)` –

Normalization mode. For the forward transform (`rfft2()`), these correspond to:

"forward" - normalize by $1/n$

"backward" - no normalization

"ortho" - normalize by $1/\sqrt{n}$ (making the real FFT orthonormal)

Where $n = \text{prod}(s)$ is the logical FFT size. Calling the backward transform (`irfft2()`) with the same normalization mode will apply an overall normalization of $1/n$ between the two transforms. This is required to make `irfft2()` the exact inverse.

Default is "backward" (no normalization).

`out (Tensor, optional)` – the output tensor.

Compared against the full output from `fft2()`, we have all elements up to the Nyquist frequency.

The discrete Fourier transform is separable, so `rfft2()` here is equivalent to a combination of `fft()` and `rfft()`: